MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 29 — Typage Statique et Dataclasses

type hints, mypy, dataclasses — écrire du Python robuste et lisible

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

27 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 29 — Typage Statique et Dataclasses **2**
0.1 Les annotations de type — Type Hints 2
0.2 Le module typing — Types avancés 3
0.3 mypy — Vérification statique des types 5
0.4 dataclasses — Classes de données sans boilerplate 6
0.5 Dataclasses avancées — frozen, order, slots 7
0.6 Generic et Protocol — Typage avancé 9
0.7 Ce qui m'a le plus surpris dans ce chapitre 11
0.8 Résumé du Chapitre 29 11

Chapitre 29 — Typage Statique et Dataclasses

À retenir

Python est dynamiquement typé — mais depuis Python 3.5, il supporte les **annotations de type** qui permettent un typage optionnel. Ces annotations rendent le code plus **lisible**, facilitent la **détection d'erreurs** avec `mypy` avant l'exécution, et améliorent l'**autocomplétion** dans les IDEs. Les **dataclasses** réduisent le boilerplate des classes de données. Ensemble, ces outils élèvent la qualité du code Python au niveau des langages statiquement typés — sans perdre la flexibilité de Python.

Les annotations de type — Type Hints

Type hints — annoter les fonctions

```python
# Sans annotations      ambigu
def calculer(notes, poids):
    return sum(n * p for n, p in zip(notes, poids))

# Avec annotations      explicite et documente
def calculer_moyenne_ponderee(
    notes:  list[float],
    poids:  list[float]
) -> float:
    """Calcule la moyenne pond ree d'une liste de notes."""
    if len(notes) != len(poids):
        raise ValueError("Longueurs incompatibles")
    total_poids = sum(poids)
    if total_poids == 0:
        raise ValueError("Somme des poids nulle")
    return sum(n * p for n, p in zip(notes, poids)) /
       total_poids

# Annoter les variables
prenom:     str    = "Alice"
age:        int    = 25
note:       float  = 15.5
admis:      bool   = True
prenoms:    list[str]         = ["Alice", "Bob"]
scores:     dict[str, float]  = {"Alice": 15.5}
paire:      tuple[int, str]   = (1, "premier")

# Annoter une classe
class Etudiant:
    prenom:  str
```

```
    note:    float
    ville:   str
    actif:   bool = True

    def __init__(self, prenom: str,
                 note: float, ville: str) -> None:
        self.prenom = prenom
        self.note   = note
        self.ville  = ville

    def mention(self) -> str:
        if self.note >= 16: return "TB"
        if self.note >= 14: return "B"
        if self.note >= 12: return "AB"
        if self.note >= 10: return "P"
        return "F"

    def __repr__(self) -> str:
        return (f"Etudiant(prenom={self.prenom!r}, "
                f"note={self.note})")
```

Le module typing — Types avancés

typing — Optional, Union, Callable, TypeVar

```
from typing import (Optional, Union, Callable,
                    TypeVar, Generic, Any,
                    TypedDict, Protocol)

# Optional     peut etre None
def chercher_etudiant(email: str) -> Optional[dict]:
    """Retourne l'etudiant ou None s'il n'existe pas."""
    base = {"alice@ensea.ci": {"prenom": "Alice", "note": 15}}
    return base.get(email)

# Union     plusieurs types possibles (Python 3.10+: X | Y)
def formater_note(note: Union[int, float]) -> str:
    return f"{float(note):.1f}/20"

# Python 3.10+ : syntaxe simplifiee avec |
def formater_note_v2(note: int | float) -> str:
    return f"{float(note):.1f}/20"

# Callable     type d'une fonction
def appliquer(
    valeurs: list[float],
    fn:      Callable[[float], float]
) -> list[float]:
```

```python
    return [fn(v) for v in valeurs]

# Resultat
print(appliquer([1.0, 4.0, 9.0], lambda x: x**0.5))
# [1.0, 2.0, 3.0]

# TypeVar      genericite
T = TypeVar("T")

def premier(liste: list[T]) -> Optional[T]:
    """Retourne le premier element ou None."""
    return liste[0] if liste else None

print(premier([1, 2, 3]))       # 1  (int infere)
print(premier(["a", "b"]))      # 'a' (str infere)
print(premier([]))              # None

# TypedDict      dict avec structure fixe
class ProfilEtudiant(TypedDict):
    prenom:  str
    note:    float
    ville:   str
    actif:   bool

def creer_profil(prenom: str, note: float,
                 ville: str) -> ProfilEtudiant:
    return {
        "prenom": prenom,
        "note":   note,
        "ville":  ville,
        "actif":  True
    }

profil = creer_profil("Alice", 15.5, "Abidjan")
print(profil["prenom"])   # mypy verifie l'acces !

# Protocol      duck typing type
class Comparable(Protocol):
    def __lt__(self, other: Any) -> bool: ...
    def __le__(self, other: Any) -> bool: ...

def trier(items: list[Comparable]) -> list[Comparable]:
    return sorted(items)
```

mypy — Vérification statique des types

mypy — détecter les erreurs avant l'exécution

```python
# pip install mypy
# mypy mon_script.py
# mypy --strict mon_script.py        mode strict

# Exemples d'erreurs detectees par mypy :

# 1. Type incompatible
def doubler(n: int) -> int:
    return n * 2

resultat: int = doubler("hello")
# mypy: error: Argument 1 to "doubler" has
#              incompatible type "str"; expected "int"

# 2. Retour incorrect
def get_note(email: str) -> float:
    if email == "alice@ci":
        return 15.5
    # mypy: error: Missing return statement

# 3. Attribut inexistant
class Config:
    host: str = "localhost"
    port: int = 8000

cfg = Config()
print(cfg.timeout)
# mypy: error: "Config" has no attribute "timeout"

# 4. None non gere
def chercher(cle: str) -> Optional[str]:
    data = {"a": "alpha"}
    return data.get(cle)

valeur = chercher("a")
print(valeur.upper())   # ERREUR si valeur est None !
# mypy: error: Item "None" of "Optional[str]" has no
#              attribute "upper"

# Correction propre
valeur = chercher("a")
if valeur is not None:
    print(valeur.upper())   # mypy est satisfait

# Configuration mypy (mypy.ini ou pyproject.toml)
# [mypy]
```

```
# python_version = 3.11
# strict = true
# warn_return_any = true
# warn_unused_imports = true
# disallow_untyped_defs = true
```

dataclasses — Classes de données sans boilerplate

@dataclass — générer automatiquement *init*

```
from dataclasses import dataclass, field, asdict, astuple
from typing import ClassVar, Optional
from datetime import datetime

# Sans dataclass     beaucoup de boilerplate
class EtudiantManuel:
    def __init__(self, prenom: str, note: float,
                 ville: str = "Inconnue") -> None:
        self.prenom = prenom
        self.note   = note
        self.ville  = ville

    def __repr__(self) -> str:
        return (f"EtudiantManuel(prenom={self.prenom!r}, "
                f"note={self.note}, ville={self.ville!r})")

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, EtudiantManuel):
            return False
        return (self.prenom == other.prenom and
                self.note   == other.note)

# Avec dataclass     tout genere automatiquement !
@dataclass
class Etudiant:
    prenom:  str
    note:    float
    ville:   str    = "Inconnue"
    actif:   bool   = True

    # field() pour les valeurs par defaut complexes
    notes_hist: list[float] = field(default_factory=list)

    # Attribut de classe     pas un champ
    NB_ETUDIANTS: ClassVar[int] = 0

    def __post_init__(self) -> None:
        """Validation apr s __init__ automatique."""
```

```
        Etudiant.NB_ETUDIANTS += 1
        if not 0 <= self.note <= 20:
            raise ValueError(f"Note invalide : {self.note}")
        self.prenom = self.prenom.capitalize()

    @property
    def mention(self) -> str:
        if self.note >= 16: return "Tres Bien"
        if self.note >= 14: return "Bien"
        if self.note >= 12: return "Assez Bien"
        if self.note >= 10: return "Passable"
        return " chec "

    def ajouter_note(self, note: float) -> None:
        self.notes_hist.append(note)
        self.note = sum(self.notes_hist) / len(self.notes_hist)


# Utilisation      __init__, __repr__, __eq__ gratuits !
alice = Etudiant(prenom="alice", note=15.5, ville="Abidjan")
bob   = Etudiant("bob", 18.0)
eve   = Etudiant("eve", 15.5)

print(alice)         # Etudiant(prenom='Alice', note=15.5, ...)
print(alice == eve) # True  (memes valeurs)
print(alice.mention)  # Passable     Assez Bien

# Conversion facile
print(asdict(alice))    #     dict Python
print(astuple(alice))   #     tuple Python

alice.ajouter_note(17.0)
print(f"Nouvelle moyenne : {alice.note:.1f}")

print(f"Total   tudiants   : {Etudiant.NB_ETUDIANTS}")
```

Dataclasses avancées — frozen, order, slots

Options avancées des dataclasses

```
from dataclasses import dataclass, field
from typing import Optional

# frozen=True      immuable (comme un namedtuple)
@dataclass(frozen=True)
class Point:
    x: float
    y: float
```

```
    z: float = 0.0

    def distance(self, autre: "Point") -> float:
        return ((self.x - autre.x)**2 +
                (self.y - autre.y)**2 +
                (self.z - autre.z)**2) ** 0.5

p1 = Point(1.0, 2.0)
p2 = Point(4.0, 6.0)
print(f"Distance : {p1.distance(p2):.2f}")

# Hashable car frozen     peut etre cle de dict ou dans set
positions = {p1, p2}
cache = {p1: "origine proche"}

# order=True     permet la comparaison < > <= >=
@dataclass(order=True)
class Note:
    # sort_index doit  tre  PREMIER pour order
    valeur:   float
    matiere:  str   = field(compare=False)
    etudiant: str   = field(compare=False)

notes = [
    Note(15.5, "Python",  "Alice"),
    Note(18.0, "Stats",   "Bob"),
    Note(12.0, "Python",  "Charlie"),
]
notes_triees = sorted(notes)
for n in notes_triees:
    print(f"  {n.valeur:5.1f} | {n.matiere} | {n.etudiant}")

# slots=True (Python 3.10+)     memoire optimisee
@dataclass(slots=True)
class Vecteur:
    x: float
    y: float

    def norme(self) -> float:
        return (self.x**2 + self.y**2) ** 0.5

    def __add__(self, autre: "Vecteur") -> "Vecteur":
        return Vecteur(self.x + autre.x,
                       self.y + autre.y)

v1 = Vecteur(3.0, 4.0)
v2 = Vecteur(1.0, 0.0)
print(f"||v1|| = {v1.norme()}")
print(f"v1+v2  = {v1 + v2}")
```

```python
# Heritage de dataclass
@dataclass
class Personne:
    prenom: str
    nom:    str
    age:    int

@dataclass
class EtudiantAvance(Personne):
    filiere:    str
    annee:      int   = 1
    note_moy:   float = 0.0

    @property
    def nom_complet(self) -> str:
        return f"{self.prenom} {self.nom}"

e = EtudiantAvance("Alice","Dupont",22,"AS",2,15.5)
print(e.nom_complet)
print(e)
```

Generic et Protocol — Typage avancé

Generic — classes paramétrées par le type

```python
from typing import TypeVar, Generic, Optional, Iterator
from dataclasses import dataclass, field

T = TypeVar("T")
K = TypeVar("K")
V = TypeVar("V")

# Classe generique     fonctionne avec n'importe quel type
@dataclass
class Pile(Generic[T]):
    """Pile LIFO generique et typee."""
    _elements: list[T] = field(default_factory=list)

    def empiler(self, element: T) -> None:
        self._elements.append(element)

    def depiler(self) -> T:
        if self.est_vide():
            raise IndexError("Pile vide")
        return self._elements.pop()

    def sommet(self) -> Optional[T]:
        return self._elements[-1] if self._elements else None
```

```python

    def est_vide(self) -> bool:
        return len(self._elements) == 0

    def __len__(self) -> int:
        return len(self._elements)

    def __iter__(self) -> Iterator[T]:
        return iter(reversed(self._elements))


# mypy sait que c'est une Pile[int]
pile_entiers: Pile[int] = Pile()
pile_entiers.empiler(1)
pile_entiers.empiler(2)
pile_entiers.empiler(3)
print(f"Sommet : {pile_entiers.sommet()}")    # 3
print(f"Taille : {len(pile_entiers)}")        # 3

# mypy sait que c'est une Pile[str]
pile_mots: Pile[str] = Pile()
pile_mots.empiler("Python")
pile_mots.empiler("est")
pile_mots.empiler("g nial")

# Protocol     interface structurelle (duck typing typ )
from typing import Protocol, runtime_checkable

@runtime_checkable
class Serialisable(Protocol):
    def to_dict(self) -> dict: ...
    def to_json(self) -> str: ...

@dataclass
class Resultat:
    etudiant: str
    note:     float

    def to_dict(self) -> dict:
        return {"etudiant": self.etudiant,
                "note":     self.note}

    def to_json(self) -> str:
        import json
        return json.dumps(self.to_dict())

def exporter(obj: Serialisable) -> str:
    return obj.to_json()

r = Resultat("Alice", 15.5)
```

```
print(isinstance(r, Serialisable))    # True
print(exporter(r))
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Les type hints sont de la documentation exécutable. Au départ, annoter `def f(x: int) -> str:` me semblait redondant avec la docstring. J'ai compris leur vraie valeur quand `mypy` a détecté une erreur que j'aurais mise des heures à trouver à l'exécution : une fonction retournant `None` implicitement alors que le type déclaré était `str`. Le typage statique n'est pas une contrainte — c'est un filet de sécurité qui s'active avant même de lancer le programme.

2. `@dataclass` génère dix méthodes en une ligne. Écrire une classe avec `__init__`, `__repr__`, `__eq__`, `__hash__` manuellement pour chaque structure de données est fastidieux et source d'erreurs. `@dataclass` les génère toutes automatiquement depuis les annotations de champs. Avec `frozen=True`, on obtient en plus `__hash__` et l'immuabilité. C'est l'équivalent pythonique des *value objects* du Domain Driven Design.

3. `Protocol` réconcilie duck typing et typage statique. Python a toujours favorisé le duck typing — si ça ressemble à un canard et cancane comme un canard, c'est un canard. `Protocol` formalise cette philosophie : au lieu d'exiger qu'une classe hérite d'une interface, on déclare quelles méthodes sont nécessaires. Une classe implémente automatiquement un `Protocol` si elle a les bonnes méthodes — sans `isinstance` ni héritage explicite. C'est la flexibilité de Python avec la rigueur du typage.

Résumé du Chapitre 29

Ce que j'ai appris dans ce chapitre :

- **Type hints** : annoter variables, paramètres et retours de fonctions.
- `Optional[T]`, `Union[X,Y]`, `T | Y` (3.10+) — types composés.
- `Callable`, `TypeVar`, `TypedDict` — types avancés du module `typing`.
- **mypy** — vérification statique avant exécution : `mypy script.py`.
- `@dataclass` — générer `__init__`, `__repr__`, `__eq__` automatiquement.
- `field(default_factory=list)` — valeurs par défaut mutables sûres.
- `frozen=True`, `order=True`, `slots=True` — options avancées.
- `Generic[T]` — classes paramétrées par le type.

`Protocol` — interfaces structurelles, duck typing typé.

"Python dynamique et Python typé ne sont pas opposés. Les type hints apportent la rigueur sans sacrifier la flexibilité. C'est le meilleur des deux mondes."